



Optimized Hackathon Ideas for Podman

Desktop

These projects are designed to leverage existing **Podman AI Lab Recipes** and Red Hat's **Granite models** to ensure they are runnable in a 24-hour window.

1. The "Open-Source Academic" (RAG-Based)

- **Original Idea:** *Research Paper Assistant / Literature Mapping*
- **Hackathon Goal:** Build a local, private research assistant that doesn't leak sensitive thesis data to the public cloud.
- **Podman Implementation:** Use the **Local RAG Recipe**.
 - **The Hack:** Students can "fork" the RAG recipe's frontend to add "theme clustering" or "gap analysis" logic on top of the retrieved documents.
 - **Hint:** Point teams to the **Granite-7b-instruct** model—it's optimized for following complex task instructions like summarizing academic text.

2. The "Intelligent DevOps Partner" (Code-Based)

- **Original Idea:** *Codebase Onboarding / Test Generation / Debugging Assistant*
- **Hackathon Goal:** Create a local CLI tool or simple web app that "explains" a repository's structure or generates unit tests without sending code to an external API.
- **Podman Implementation:** Use the **Code Generation Recipe**.
 - **The Hack:** Instead of a generic chatbot, students build a wrapper that feeds local file structures into the model.
 - **Hint:** Use **Granite-20b-code** or **Granite-8b-code**. These are specifically trained on 100+ languages to handle enterprise-level software tasks.

3. The "Granite Tutor" (Agentic Workflow)

- **Original Idea:** *Course Companion / LLM Coding Tutor*
- **Hackathon Goal:** A step-by-step learning agent that doesn't just give answers but asks "clarification questions" to guide the student.
- **Podman Implementation:** Use the **Inference Server** (OpenAI-compatible API).
 - **The Hack:** Use a Python/FastAPI backend to maintain "State." The agent can "wait" for user input before revealing the next study step.
 - **Hint:** Use the **Chatbot Recipe** as a starting point for the UI, then modify the backend to inject "System Prompts" that force the model to act as a teacher, not a calculator.

4. The "Literature Detective" (Clustering & Mapping)

- **Original Idea:** *Literature Mapping / Benchmark Discovery*
- **Hackathon Goal:** A tool where a student uploads 10-15 PDFs, and the agent identifies the "missing link" or the "open gap" across all of them, suggesting a specific benchmark to test.
- **Podman Implementation:** Use the **Local RAG Recipe** + a custom **Python script**.

- **The Hack:** Instead of just a "chat with PDF," students implement a "Compare & Contrast" prompt template. The agent reads all ingested files and outputs a Markdown table comparing their methods and benchmarks.
- **Hint:** Use **Granite-7b-instruct**. Its strong performance in summarization makes it ideal for extracting structured data (like metrics and baselines) from messy academic text.

5. The "Paper-to-Pitch" Creator (Multimodal-lite)

- **Original Idea:** *Presentation Builder Agent / Academic Writing Assistant*
- **Hackathon Goal:** Transform a complex GitHub README or a Research Paper into a structured 5-slide outline with "Speaker Notes" tailored for a specific audience (e.g., a professor vs. a VC).
- **Podman Implementation:** Use the **Inference Server** + a frontend UI (like Streamlit).
 - **The Hack:** Use the agent to generate JSON-formatted slide content. Students can use a simple Python library (like python-pptx) to turn that JSON into an actual downloadable PowerPoint file.
 - **Hint:** Point teams toward **Mistral-7b-instruct** for creative writing tasks or **Granite-7b** for technical accuracy. Suggest they experiment with "Few-Shot Prompting" to show the model examples of good slide structures.

6. The "Experiment Log Scientist" (Data Analysis)

- **Original Idea:** *Experiment Log Agent / Baseline Runner*
- **Hackathon Goal:** A "Lab Manager" agent that sits in a directory, reads CSV/Log files from a model training run, and writes a "Post-Mortem" report explaining why the model might be overfitting or underperforming.
- **Podman Implementation:** Use the **Inference Server** (OpenAI-compatible API).
 - **The Hack:** Use a "ReAct" (Reasoning + Acting) pattern. The student writes a small script that lets the LLM "see" the first 10 lines of a CSV, ask for a specific column, and then perform the analysis.
 - **Hint:** Use **Granite-20b-code**. Even though this is a "data" task, the code-specific model is excellent at understanding structured data formats like CSVs and JSON logs.

7. The "Smart Onboarding" Bot (Repo Explorer)

- **Original Idea:** *Codebase Onboarding / Debugging Assistant*
- **Hackathon Goal:** A tool that indexes a local Git repository and answers: "Where is the logic for X?" or "How do I add a new API endpoint based on the existing pattern?"
- **Podman Implementation:** Use the **Code Generation Recipe**.
 - **The Hack:** Instead of just generating new code, students use a "Context Injection" strategy. They can use a tool like tree to feed the file structure to the model so the agent has a "map" of the project.
 - **Hint:** Use **Granite-8b-code**. It is small enough to keep the entire project structure in its context window without crashing the student's RAM, making it very responsive for navigation tasks.

The Student Cheat Sheet: Recipes & Models

This is what your associates should point students to during the first few hours of the hackathon.

Project Type	Recommended Recipe	Best Model (Local CPU)	Why?
Research & Docs	RAG Chatbot	granite-3.3-8b-instruct	Best-in-class for RAG and technical summarization.
Coding & Git	Code Generation	granite-8b-code	High performance on 100+ languages; fits in 16GB RAM.
Data & Logs	Inference Server	granite-20b-code	Surprisingly good at reasoning through structured JSON/CSV logs.
Creative/Agentic	Chatbot	mistral-7b-instruct	Great "personality" and diverse instruction-following.

Expert "Associate" Advice for Teams

1. Fix the "Cold Start" (Resource Setup)

- **The Advice:** "Don't just start the recipe."
- **The Fix:** Before starting, go to **Podman Desktop Settings > Podman Machine** and set your resources to at least **6 CPUs and 12GB RAM**. If you leave it at the default

(2GB), the LLM will crash the container immediately.

2. The "Local Secret" (Privacy Advantage)

- **The Advice:** "Use this in your pitch!"
- **The Pitch:** Remind judges that because you are using Podman AI Lab, your research data (PDFs/Code) **never leaves your machine**. This is how Enterprise AI (Red Hat's bread and butter) actually works in regulated industries.

3. Model Speed vs. Quality (Quantization)

- **The Advice:** "Pick the right file."
- **The Fix:** When downloading from the catalog, look for **GGUF** format with **Q4_K_M** quantization. It's the "Goldilocks" setting—small enough to be fast, but smart enough to win the hackathon.

4. Peek Under the Hood

- **The Advice:** "Don't just use the UI."
- **The Fix:** Every recipe in Podman AI Lab is just a set of containers. Students can click **"Open in VS Code"** directly from the recipe page to modify the Python/Streamlit code. This is where the real "hacking" happens.

 Implementation Toolkit: Models & Recipes

To get students started quickly, provide them with this "Cheat Sheet":

Project Type	Recommended Recipe	Suggested Model	Why?
Research/RAG	RAG Chatbot	granite-7b-instruct	High instruction-following accuracy.
Coding/DevOps	Code Generation	granite-8b-code	Optimized for speed and low RAM on laptops.

Multi-Agent	Inference Server	mistral-7b-instruct	Great generalist model for orchestrating tasks.
-------------	------------------	---------------------	---



3. Pro-Hacker Tips

- **Avoid the "WiFi Death":** Don't wait until the hackathon starts to download models. They are multi-GB files. Download your base models (Granite 7B/8B) the night before.
- **The "Context" Secret:** LLMs have a "limit" on how much text they can remember. If you are analyzing a paper, "chunk" it into sections instead of pasting the whole 50 pages at once.
- **Open in VS Code:** Every Podman AI Lab recipe has a button to **"Open in VS Code."** Use this to customize the UI and logic—don't just use the default chatbot!
- **API Compatibility:** Podman AI Lab provides an **OpenAI-compatible API**. This means you can use almost any existing Python AI library (like LangChain) just by changing the `base_url` to your local machine.



Quick Tips for your Associates to share:

- **The "Quantization" Trick:** If a student's laptop is lagging, tell them to look for the **4-bit (Q4_K_M)** version of the models in the Podman AI Lab catalog. It reduces RAM usage by nearly 50% with almost no loss in "intelligence" for these tasks.
- **Context Window Management:** Remind students that for "Academic Writing" tasks, they shouldn't dump a whole 30-page paper at once. Teach them to "chunk" the data—this is a core Red Hat/OpenShift AI concept they can learn during the hackathon.
- **Local API = Privacy:** Emphasize that because they are using **Podman AI Lab**, they can "leak" their smartest, most secret research ideas into the model, and it *never* leaves their machine. This is a huge selling point for university researchers.



Pro-Tips for the Organizers

Avoid "Compute Traps"

- **Remove "Neural Network Compressor":** This is too heavy for most student laptops. Instead, suggest they use **Pre-Quantized models** (GGUF format) from the Podman AI Lab catalog to show they understand "efficiency".
- **Limit Concurrent Models:** Advise students not to run more than **3 models** simultaneously to prevent their laptops from freezing.

Shared Resource "War Room"

- **Local Model Registry:** Since download speeds will be an issue, set up a local server where students can curl the models over the local network instead of the internet.
- **Podman Machine Specs:** Remind students to increase their Podman Machine resources to at least **12GB RAM and 6 CPUs** before starting recipes, or the containers will crash.

Course Companion Agent

An agent that helps students learn from course materials. Students upload slides, notes, or readings, and the agent produces explanations, summaries, quizzes, and guided study paths. A strong version could identify weak points and suggest what to review next.

Agentic Cyber-Sec: Autonomous Vulnerability Detection and Auto-Patching

This project introduces an autonomous cybersecurity agent that acts as both an attacker (Red Team) and a defender (Blue Team) in a continuous loop. The agent actively probes a designated web application by executing common exploits, such as SQL Injection. Upon successfully breaching the system, the agent analyzes the vulnerable source code and autonomously injects a protective patch, such as applying parameterized queries. Finally, the agent re-runs the initial attack to verify that the vulnerability is neutralized, generating a detailed report of the attack vector and the applied defense mechanism.

Research Paper Assistant

An agent that reads academic papers and extracts the main problem, method, assumptions, results, limitations, and possible follow-up directions. This is suitable for students who want to quickly understand unfamiliar literature.

Literature Mapping Agent

An agent that helps build a small literature survey around a topic. Given a research question, it collects or receives papers, clusters them into themes, identifies key methods, and summarizes open gaps.

Benchmark Discovery Agent

An agent that helps researchers find suitable datasets, baselines, and metrics for a given problem. For example, a user could describe a task, and the agent would suggest relevant benchmarks, standard models, evaluation metrics, and an initial experimental plan.

Baseline Runner Agent

An agent that turns a research idea into a runnable baseline experiment. The agent generates configuration files, training scripts, evaluation code, and a short report template. A 24-hour version can focus on one domain, such as tabular data, NLP, or computer vision.

Codebase Onboarding Agent

An agent that helps new users understand an unfamiliar repository. It explains the project structure, identifies the main files, describes how to install and run the code, and suggests a first contribution task.

Test Generation Agent

An agent that reads a function or small codebase and generates unit tests, edge cases, and simple documentation. A stronger version can run the tests and explain failures.

Debugging Assistant Agent

An agent that receives an error message, relevant code, and execution context, then proposes a debugging plan and possible fixes. The project can focus on making the agent ask good clarification questions and explain its reasoning clearly.

LLM Coding Tutor

An agent that teaches students how to use LLMs effectively for programming. It guides them through a coding task step by step, suggests useful prompts, checks generated code, creates tests, and explains mistakes.

Experiment Log Agent

An agent that helps organize research experiments. It summarizes runs, compares results, detects missing baselines, and produces a clean experiment report from logs or CSV files.

Academic Writing Assistant

An agent that helps improve technical writing. It can rewrite abstracts, related-work sections, reviewer responses, or grant-style summaries while preserving the original meaning and technical claims.

Presentation Builder Agent

An agent that turns a paper, project, or code repository into a short presentation. It proposes the story, slide titles, main figures, and speaker notes.

Neural Network Compressor/Trainer Agent

An agent that helps users train and optimize neural networks efficiently. Given a dataset and task, it proposes a simple architecture, trains a model, and applies lightweight compression techniques such as pruning, quantization, or distillation. It can also compare performance before and after compression and suggest improvements.

Food & Gut Data Agent

An agent that helps users explore whether the foods people eat influence the gut microbiome over time. Given food logs, repeated microbiome measurements, and a reference dataset describing microbes found in different foods, the agent maps foods to microbial profiles, computes a daily food-microbe exposure score, and tests whether that exposure is associated with gut stability, diversity, and overall community structure.